

Food Store — TPI

Entrega parcial · Unidades 1-3

Base de Datos II — UTN · Equipo G · 22/09/2026

Integridad, transacciones y concurrencia (U1) · Optimización de consultas (U2)

Índices, vistas y objetos programables en PL/pgSQL (U3)

Motor: PostgreSQL 16+ · ENUM, TIMESTAMPTZ, IDENTITY, JSONB, CALL, tablas de transición

Documentos que abren la entrega: informe técnico y checklist de objetivos 1-9

Informe técnico — Entrega parcial del TPI «Food Store»

Base de Datos II · UTN · Equipo G · 22/09/2026 · Motor: PostgreSQL 18 (≥16) + PL/pgSQL

Resumen de lo actuado en las Unidades 1–3 con el flujo de cátedra **Kiro (specs)** → **OpenCode (IA)** → **el motor decide (evidencia)**. El checklist de verificación por objetivo está en `docs/checklist_TPI.md` y la bitácora de uso de IA en `duia.md`.

Unidad 1 — Integridad, transacciones y concurrencia

Qué se implementó

- Modelo ER → relacional → normalización a BCNF → DDL definitivo (`schema.sql`), con ENUM, `TIMESTAMPZ`, `IDENTITY`, PK/FK con `ON DELETE RESTRICT`, `UNIQUE` (email, nombre de categoría, par de detalle), `CHECK` y 3 índices justificados.
- Reglas R8–R10 garantizadas por el motor con triggers PL/pgSQL (`sql/restricciones.sql`): transiciones de estado del pedido, stock suficiente con descuento atómico y precio de línea congelado.
- Protocolo de trabajo seguro sobre copia: `protocolo_seguridad.md` (copia → transacción → respaldo).

Cómo se probó

- DDL: ejecución completa de `schema.sql` sobre una copia de trabajo, sin errores.
- Triggers: batería de casos **válidos** (INSERT/UPDATE lícitos dentro de la transacción, COMMIT final) e **inválidos** (cada uno en su transacción que se aborta y ROLLBACK); se verificó el mensaje de excepción de cada regla.
- Concurrencia: 4 escenarios reproducidos con **dos sesiones** `psql` simulando dos terminales (informe `docs/informe_concurrencia.md`).

Resultados

- READ COMMITTED permite **lectura no repetible** (precio 1200→1300) y **fantasma** (COUNT 2→3); REPEATABLE READ los elimina con instantánea fija.
- Un `UPDATE` de la misma fila se **bloqueó 4,51 s** hasta el COMMIT de la otra sesión (mecanismo que evita la pérdida de la actualización); el conflicto se abortó con `40P01`.
- Los casos inválidos fueron rechazados por el motor con el mensaje de la regla correspondiente (R8 transición, R9 sobreventa, R10 precio congelado).

Unidad 2 — Optimización de consultas

Qué se implementó

- Carga masiva de datos para explotar planes no triviales (`sql/carga_masiva.sql` TP3): 50 k productos, 20 k clientes, 200 k pedidos + 512 k líneas.
- 5 consultas analíticas de 3+ tablas (`sql/queries_tp4.sql`) y su versión optimizada por nodo (`sql/queries_tp4_opt.sql`): vista materializada, `work_mem`, reescritura y funciones de ventana.
- Equivalencias verificadas entre versiones distintas y entre funciones de ventana / subconsultas (`sql/equivalencias_tp4.sql`).

Cómo se probó

- `EXPLAIN (ANALYZE, BUFFERS, TIMING)` antes/después por consulta, planes archivados en `docs/planes4/*.txt` ; lecturas críticas de planes en `docs/ejercicio_lectura_critica_join.md` (inversión externa/interna del Nested Loop, Memoize que cachea cliente pero no pedidos, cost ≠ tiempo).
- Equivalencias con `EXCEPT` → 0 filas (parte del enunciado).

Resultados (antes → después, best of)

Consulta	Técnica	Antes	Después	Ganancia
T1 facturación por categoría/mes	MV <code>mv_facturacion_categoria_mes</code>	1.101 ms	0,6 ms	×1.846
T2 top 3 por categoría (window)	<code>work_mem</code> + plan	125 s	29 s	×4,3
T3 top 50 pedidos	reescritura alias	78,1 ms	47,9 ms	×1,63
T5 acumulado mensual por cliente	MV	6.593 ms	1.887 ms	×3,5
Q6 competencia (TP3)	índices propuestos por IA	—	—	×42

Se **descartaron** propuestas de IA que no mejoraban o rompían equivalencia (OPT-T5-A y la variante con `COUNT(DISTINCT)` que ocupaba DENSE_RANK: se detectó por la diferencia 19.743 y se corrigió a `COUNT(*)`).

Unidad 3 — Índices, vistas y objetos programables del motor

Qué se implementó

- Índices con justificación de plan previo (`sql/indices.sql` , `specs/`), incluido un **índice parcial** para reposición de stock.
- 4 vistas (`sql/views.sql`): 3 de reporte + 1 **de seguridad** que oculta datos personales (no expone `telefono`).
- Vista materializada con índice único para `REFRESH CONCURRENTLY` (`sql/materializadas.sql`).
- **Nuevo en esta entrega:** funciones de negocio PL/pgSQL, función que produce **JSONB**, procedimiento invocado con **CALL** y trigger a nivel sentencia con **tabla de transición** (`sql/procedimientos.sql`).

Cómo se probó

- Mediciones `EXPLAIN (ANALYZE)` antes/después (best of 3), prueba de **costo de escritura** (6.000 pedidos + 6.000 líneas) y descarte D1 de sobreindexación verificado en el motor (el plan seguía siendo Seq Scan).
- Vistas: equivalencia contra la consulta manual con `EXCEPT` → 0/0 en las 4 y verificación de que `v_cliente_contacto` no tiene `telefono`.
- MV: equivalencia 0/0, build y refresco concurrente; procedimientos/función/ transición: `sql/prueba_procedimientos_TPI.sql` (CALL, funciones, auditoría).
- La **repetición de corridas** confirmó estabilidad (no single-shot).

Resultados

Ítem	Antes	Después	Diferencia
A1 pedidos del mes ((fecha, estado))	18,8 ms	11,5 ms	×1,6
A2 stock bajo (índice parcial)	10,4 ms	0,34 ms	×30,4
A3 cliente por apellido (opclass)	2,75 ms	1,40 ms	×2,0
Escritura pedido (costo real)	234,5 ms	237,8 ms	+1,4 %
MV facturación ×41.000	905,8 ms	0,022 ms	×41.000
Descartes	D1 (cantidad) y D2 (cliente, estado)	—	documentados en specs/

Vista materializada: `REFRESH CONCURRENTLY` habilitado por índice único, frecuencia justificada (corte mensual, 117 filas → refresco económico).

Uso de herramientas de IA

- **IA usada:** solo **OpenCode** (la herramienta indicada por la cátedra), con el flujo de especificación de **Kiro**. No se utilizó ninguna otra herramienta de IA.
- **Decisión aceptadas** (con verificación en el motor): índices A1–A3, MVs T1/T5, reescritura de T3, procedimiento/funciones y trigger de transición propuestos por la IA y validados con EXPLAIN/equivalencia.
- **Decisiones descartadas:** OPT-T5-A (equivalencia), índice D1 `detalle(cantidad)` (el plan no cambiaba), D2 redundante, y la especificación inicial de la parte 3 de TP4 que usaba `COUNT(DISTINCT)` como DENSE_RANK (corregida con `COUNT(*)`).
- **Verificación humana:** cada script se aplicó sobre la **copia de trabajo** dentro de `BEGIN/COMMIT` o `BEGIN/ROLLBACK`, sin tocar las bases en uso; los planes y números provienen de corridas reales del motor, no de estimaciones.

Reproducción completa y comandos: `README.md`. Evidencia por objetivo: `docs/checklist_TPI.md`.
Bitácora de sesiones IA: `duia.md`.

Checklist TPI — Food Store · Base de Datos II (UTN)

Verificación de cobertura de los 9 objetivos de la entrega parcial (Unidades 1-3). Para cada objetivo: **evidencia** (archivo) y **cómo verificarlo**. Marcar ✓ al presenciar la corrida.

Motor: PostgreSQL 18 (≥ 16 exigido) con PL/pgSQL. Base de verificación: `food_store_tp5` (creada por copia de `food_store_tp3`).

1. Modelo ER (entidades, atributos, claves, cardinalidad, participación)

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 1) · `TP1/diagrama_ER.png` · reglas R1–R7 en `schema.sql`.
- **Verificar:** entidades Cliente/Categoría/Producto/Pedido con atributos y claves; cardinalidades 1:N (categoría–producto, cliente–pedido) y N:M (pedido–producto); participación total justificada por R1/R2 y parcial por R6 (mail único).

2. Paso de ER a relacional (1:N y N:M con tabla intermedia)

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 2) · `schema.sql`.
- **Verificar:** 1:N con FK en el lado N (`producto.categoria_id`, `pedido.cliente_id`); N:M resuelta con tabla asociativa `detalle_pedido` (FK doble + UNIQUE (`pedido_id`, `producto_id`)); clave sustituta `GENERATED ALWAYS AS IDENTITY` justificada.

3. Normalización hasta 3FN/BCNF con justificación de DF

- **Evidencia:** `TP1/desarrollo_TP1.pdf` (Parte 3) · comentarios en `schema.sql` (total derivado = DF transitiva evitada; el par (`nro_pedido`, `producto`) define cantidad y `precio_unitario`).
- **Verificar:** una sola tabla plana $\rightarrow$ 1FN $\rightarrow$ 2FN $\rightarrow$ 3FN $\rightarrow$ BCNF, con la DF (`nro_pedido`, `producto`) $\rightarrow$ (cantidad, `precio_unitario`) y la eliminación de la dependencia transitiva `nro_pedido` $\rightarrow$ cliente, `forma_pago`.

4. DDL completo (tipos, PK/FK, restricciones e índices)

- **Evidencia:** `schema.sql`.
- **Verificar:** ENUM (`forma_pago`, `estado_pedido`), `TIMESTAMPZ`, `IDENTITY`, PK, FK con `ON DELETE RESTRICT` (R7), `UNIQUE` (email, nombre de categoría, par de detalle), `CHECK` (precios/stock no negativos, cantidad positiva), 3 índices justificados (uno parcial: `WHERE activo`).

5. DML y consultas (JOIN, agregación, subconsultas, GROUP BY/HAVING, ventana)

- **Evidencia:**
- JOIN + SUM + GROUP BY: `sql/queries.sql`.
- Subconsultas + HAVING: `sql/queries_tp4.sql` (T4/T5) y `sql/equivalencias_tp4.sql`.
- Funciones de ventana: `ROW_NUMBER() OVER` y `SUM(...) OVER (PARTITION BY ... ORDER BY ...)` en `sql/queries_tp4.sql` y `sql/queries_tp4_opt.sql`.
- **Verificar:** ejecutar contra `food_store_tp5`; las equivalencias ventana↔subconsulta ya fueron validadas con `EXCEPT` 0 filas (`sql/equivalencias_tp4.sql`, sección 3).

6. Vistas, funciones y procedimientos almacenados en PL/pgSQL

- **Evidencia:**
- Vistas: `sql/views.sql` (+ verificación `EXCEPT` 0/0 en `sql/verificacion_views.sql`).
- Funciones de negocio + procedimiento + JSONB + transición: `sql/procedimientos.sql` y `sql/prueba_procedimientos_TPI.sql`.
- Funciones de trigger R8–R10: `sql/restricciones.sql`.
- **Verificar:** `CALL registrar_reposicion(1, 20)`; `SELECT calcular_total_pedido(212006)`; `SELECT resumen_pedido_jsonb(212006)` (devuelve JSONB con líneas y total); auditoría 1 fila por sentencia en `auditoria_carga_lineas`.

7. Reglas de negocio con CHECK, UNIQUE y triggers

- **Evidencia:** CHECK/UNIQUE en `schema.sql`; triggers R8 (transiciones), R9 (stock suficiente), R10 (precio congelado) en `sql/restricciones.sql`; casos válidos e inválidos en `sql/` + informe `docs/DUIA_parte1.md`.
- **Verificar:** insertar una línea sin stock → rechazo `R9`; `UPDATE` de `estado` fuera de flujo → rechazo `R8`; `UPDATE precio_unitario` → rechazo `R10`; mail duplicado → viola `UNIQUE`.

8. Transacciones: atomicidad, COMMIT, ROLLBACK, aislamiento y concurrencia

- **Evidencia:**
- Niveles de aislamiento y anomalías: `docs/informe_concurrencia.md` (READ COMMITTED vs REPEATABLE READ; no repetible, fantasma, espera de lock 4,51 s, `40P01` y `FOR UPDATE`; 2 sesiones psql).
- BEGIN/ROLLBACK como protocolo: `docs/ejercicio_lectura_critica.md` y `protocolo_seguridad.md`.
- Atomicidad con auditoría: sección 6 de `sql/prueba_procedimientos_TPI.sql` (ROLLBACK revierte la fila de auditoría).
- **Verificar:** correr los 4 escenarios del informe con dos ventanas de `psql`; verificar `filas_auditoria_tras_rollback = filas_auditoria_antes`.

9. Borrado lógico (soft delete) y su impacto en consultas e índices

- **Evidencia:** columna `activo BOOLEAN` en `categoria` y `producto` (`schema.sql`); índice parcial `idx_producto_categoria ... WHERE activo = TRUE`; Vista `v_productos_vigentes` que filtra `activo` (`sql/views.sql`); reglas R7 en comentarios del DDL.
- **Verificar:** `UPDATE producto SET activo = FALSE WHERE id = ...`; el producto desaparece de la carta (`v_productos_vigentes`) y no se puede reponer (`CALL registrar_reposicion` lo rechaza), pero el histórico de ventas se conserva (no hay DELETE físico; FKs con RESTRICT).

Nota: elementos específicos de PostgreSQL 16+ exigidos

Característica	Dónde se demuestra
ENUM	<code>schema.sql</code> (<code>forma_pago</code> , <code>estado_pedido</code>)
TIMESTAMPTZ	<code>schema.sql</code> (<code>created_at</code> , <code>pedido.fecha</code>)
Columnas IDENTITY	<code>schema.sql</code> (PK de las 5 tablas)
PL/pgSQL	<code>sql/restricciones.sql</code> , <code>sql/procedimientos.sql</code>
PL/pgSQL con JSONB	función <code>resumen_pedido_jsonb</code>
Procedimiento invocado con CALL	<code>registrar_reposicion</code>
Tablas de transición en triggers	<code>trg_detalle_auditoria_carga</code> (<code>REFERENCING NEW TABLE</code> , nivel STATEMENT)

Los contadores dependen de la copia de trabajo: los ids de pedido de la sección 6 cambian con cada corrida (21xxxx); usar `RETURNING` / `max(id)`.